ePGD Deep Learning & GenAI

End-Semester Practice Set — Solutions Report

IIT Bombay | May 2026

Complete worked solutions with explanations and revision topics for every question.

Score breakdown: Section 1 (5 MCQs × 1 = 5) | Section 2 (5 MSQs × 1 = 5) | Section 3 (5 Short Ans × 2 = 10) | Section 4 (2 Long Ans × 5 = 10) | Total: 30 marks

---

Section 1 — Single Correct Option (MCQs)

Q 1.1 — Cross-entropy and Perplexity

If model M1 achieves lower cross-entropy than M2 on a validation set, which statement must be true?

Correct Answer: A — M1 will yield a lower perplexity score than M2 on that same dataset.

Explanation:

Perplexity and cross-entropy are directly linked by the formula: **Perplexity = e^H** where H is the cross-entropy in nats (or 2^H if H is in bits). Because this is a strictly increasing exponential function, lower cross-entropy always and necessarily produces lower perplexity on the same dataset.

Why the other options are wrong:

B: Cross-entropy says nothing about the N-gram order — a trigram and a 5-gram model could have identical cross-entropy.

C: Parameter count is unrelated to cross-entropy; a small model can outperform a large one.

D: Better validation cross-entropy does not guarantee better performance on downstream generation tasks — the relationship depends on the task.

Topics to Revise
• Perplexity formula and its derivation from cross-entropy
• Relationship: MLE, log-likelihood, cross-entropy
• Why perplexity is used as an LM evaluation metric

Q 1.2 — Teacher Forcing in Seq2Seq

In a Seq2Seq model for machine translation, teacher forcing refers to:

Correct Answer: B — Feeding the ground-truth target token at step t as input to the decoder at step t+1 during training.

Explanation:

During training, instead of using the decoder's own (possibly wrong) prediction as the next input, teacher forcing feeds the actual correct token. This prevents the error accumulation problem (exposure bias) during training and stabilises/speeds up gradient updates. At inference time, the model switches to auto-regressive mode (option C) — it must use its own predictions since ground truth is unavailable.

Why option C is wrong:

C describes auto-regressive (free-running) decoding, which is used at inference — not teacher forcing.

Topics to Revise
• Teacher forcing vs. scheduled sampling
• Exposure bias problem in Seq2Seq models
• Training vs. inference mismatch in RNN decoders

Q 1.3 — BART Pre-training Objective

Which model is trained by corrupting input text and reconstructing the original via an encoder-decoder?

Correct Answer: C — BART (Bidirectional and Auto-Regressive Transformer)

Explanation:

BART is pre-trained with a denoising objective: the input is corrupted using noise functions (token masking, deletion, text infilling, sentence permutation, document rotation) and the decoder must reconstruct the original clean text. This makes BART effective for both generation and comprehension tasks.

Comparison of other models:

BERT: Encoder only. Uses MLM and NSP. No decoder — not a Seq2Seq model.

GPT: Decoder only. Autoregressive left-to-right language modeling. No corruption.

Word2Vec: Static word embeddings trained via CBOW or Skip-gram. Not an encoder-decoder.

Topics to Revise
• BART pre-training noise functions
• BERT vs GPT vs BART pre-training objectives
• Encoder-only vs decoder-only vs encoder-decoder architectures

---

Q 1.4 — Few-shot In-Context Learning

In the prompt 'Translate English to French: sea otter => loutre de mer, cheese =>', the example is used as:

Correct Answer: B — A task demonstration (few-shot in-context learning example)

Explanation:

The sea otter example shows the model the desired input-output format without any gradient updates. This is few-shot prompting / in-context learning — the model infers the task pattern from the demonstration alone. No parameters are changed; the example lives entirely in the prompt.

Why other options are wrong:

A: A reward signal belongs to RLHF/RL training, not a static prompt.

C: Learned soft prompts (prompt tuning) are trained continuous vectors, not human-readable text.

D: A validation example is used for model evaluation — not embedded in the inference prompt.

Topics to Revise

- Zero-shot vs. one-shot vs. few-shot prompting
- In-context learning (GPT-3 paper)
- Difference from fine-tuning and prompt tuning

Q 1.5 — Scaling in Scaled Dot-Product Attention

Why is the dot product in self-attention scaled by 1/sqrt(d_k) before applying softmax?

Correct Answer: C — For large d_k the dot products grow in magnitude (~sqrt(d_k)), pushing softmax into saturation regions with near-zero gradients; scaling stabilises training.

Explanation:

If Q and K vectors have components with zero mean and unit variance, their dot product has variance d_k. The standard deviation is therefore sqrt(d_k). For large d_k (e.g. 512 or 768), the dot products can be very large in magnitude, concentrating the softmax distribution near one-hot, which causes gradients to vanish. Dividing by sqrt(d_k) restores unit variance before softmax.

$$\text{Attention}(Q,K,V) = \text{softmax}\left(QK^T / \sqrt{d_k} \right) * V$$

Topics to Revise

- Derivation of scaled dot-product attention
- Softmax saturation and vanishing gradients
- Multi-head attention mechanics

Section 2 — Multiple Select Questions (MSQs)

Q 2.1 — Equivalent Objectives for N-gram LM Training

Correct Answers: A, B, D

Explanation:

These three objectives are mathematically equivalent when training a language model:

A: Maximising the log-likelihood of the corpus is the standard MLE objective.

B: Minimising cross-entropy H(p_data, p_model) = -E[log p_model] is identical to maximising log-likelihood.

D: Minimising KL(p_data || p_model) = H(p_data, p_model) - H(p_data). Since H(p_data) is a constant, this reduces to minimising cross-entropy.

C is WRONG: Maximising perplexity means maximising exp(H), which is the opposite of what we want.

Topics to Revise

- KL divergence decomposition: KL = cross-entropy - entropy
- Equivalence of MLE and cross-entropy minimisation
- Why we minimise, not maximise, perplexity

Q 2.2 — Vanilla Seq2Seq (No Attention)

Correct Answers: A, B, D

Explanation:

A (True): The encoder and decoder are separate modules — they can be heterogeneous (e.g. LSTM encoder with GRU decoder), as long as the context vector interface matches.

B (True): The entire source sentence is compressed into one fixed-size context vector. For long sequences, information is lost — this is the information bottleneck problem.

D (True): Both encoder and decoder losses flow back through a single backpropagation graph, training them jointly end-to-end.

C is WRONG: Without attention, the decoder only receives the final encoder hidden state (the context vector). It cannot access intermediate encoder states.

E is WRONG: The context vector dimension is a design hyperparameter completely independent of vocabulary size.

Topics to Revise

- Information bottleneck problem in Seq2Seq
- Encoder hidden state as context vector
- Backpropagation through time (BPTT)

Q 2.3 — Tasks Suitable for BERT-style Encoders

Correct Answers: A, B, C, E

Explanation:

BERT produces rich bidirectional contextual representations, ideal for tasks that require understanding the entire input:

A: NER — classify each token as a named entity type. Uses per-token representations.

B: Sentiment classification — uses the [CLS] token representation fed to a classifier head.

C: Extractive QA — predict start/end span positions within the context passage.

E: Sentence-pair classification (entailment, semantic similarity) — both sentences encoded together, [CLS] used.

D is WRONG: Unconditional story generation requires autoregressive decoding — generating tokens one by one. BERT's encoder architecture cannot generate text this way; GPT-style decoder-only models are suited for this.

Topics to Revise

- BERT fine-tuning tasks and their architectures
- Difference between understanding tasks and generation tasks
- GPT vs BERT architectural suitability

Q 2.4 — Orca-style Data Generation

Correct Answers: A, B, D

Explanation:

Orca (Microsoft, 2023) improves instruction-following small models by learning from rich explanatory completions of larger teacher models (GPT-4/GPT-3.5):

A: Orca improves information richness — responses include reasoning traces, not just final answers.

B: Diverse system instructions are used including: explain like I'm five (ELI5), step-by-step reasoning, chain-of-thought, helpful/detailed responses.

D: The training data uses completions generated by GPT-4 and GPT-3.5 as teacher models.

C is WRONG: Orca IS a form of instruction tuning — it is specifically designed to improve instruction-following via richer supervision signals.

Topics to Revise

- Orca and Orca-2 papers (Microsoft Research)
- Instruction tuning vs. RLHF
- Knowledge distillation from LLMs

Q 2.5 — Reinforcement Learning for LLMs

Correct Answers: A, C

Explanation:

A (True): The policy pi_theta(a|s_t) is exactly the LLM's softmax distribution over all vocabulary tokens given the current sequence (state).

C (True): The value function V(s_t) predicts expected cumulative future reward and is typically implemented as a linear head on the LLM's hidden states, trained alongside the policy.

B is WRONG: RLHF uses a SPARSE reward — a single scalar reward is provided at the end of the complete generated sequence. There is no per-token reward signal in standard RLHF.

D is WRONG: PPO (Proximal Policy Optimization) is an ON-POLICY algorithm. It uses freshly sampled data and clips the probability ratio to prevent large updates. Off-policy algorithms (like Q-learning) replay old data with importance sampling corrections.

Topics to Revise

- PPO algorithm and clipping mechanism
- Sparse vs. dense rewards in RLHF
- Value function and advantage estimation (GAE)
- On-policy vs. off-policy RL

Section 3 — Short Answer Questions (2 marks each)

Q 3.1 — Katz Backoff Weight alpha(w_{i-1})

Derive the algebraic expression for the backoff weight that ensures the conditional distribution sums to 1.

Solution:

For the distribution to sum to 1, split the sum over observed and unobserved bigrams:

$$\sum_{w_i: c>0} P^*(w_i \mid w_{i-1}) + \alpha(w_{i-1}) \cdot \sum_{w_i: c=0} P_{Katz}(w_i) = 1$$

Solving for alpha:

$$\alpha(w_{i-1}) = \frac{1 - \sum_{w_i: c(w_{i-1},w_i)>0} P^*(w_i \mid w_{i-1})}{\sum_{w_i: c(w_{i-1},w_i)=0} P_{Katz}(w_i)}$$

Intuition: The numerator is the 'leftover' probability mass not consumed by observed bigrams after Good-Turing discounting. The denominator redistributes that mass proportionally over unseen bigrams according to the unigram backoff probabilities.

Topics to Revise

- Good-Turing discounting (the P* notation)
- Katz backoff for trigrams and higher-order n-grams
- Absolute vs. Witten-Bell vs. Kneser-Ney smoothing

Q 3.2 — Information Bottleneck and Attention Mechanism

Explain the bottleneck in vanilla Seq2Seq and how attention addresses it. Describe the information flow for a 4-token source.

Solution:

The bottleneck problem: In vanilla Seq2Seq, the entire source sentence is compressed into a single fixed-size context vector (the encoder's final hidden state h_4). All decoder timesteps condition exclusively on this one vector. For long sequences, early-token information (h_1, h_2) is diluted or lost entirely as it is overwritten by later hidden states.

Architecture	Source: [w1, w2, w3, w4]	Decoder input at each step
Vanilla Seq2Seq	Encoder: h1->h2->h3->h4	Only h4 (fixed context vector)
With Attention	Encoder: h1, h2, h3, h4 (all stored)	Weighted sum $c_t = \sum_j \alpha_{t,j} \cdot h_j$
Key difference	Information bottleneck	Dynamic, position-aware context

Attention mechanism: At each decoder step t, alignment scores $e_{t,j} = \text{score}(s_{t-1}, h_j)$ are computed between the previous decoder state s_{t-1} and every encoder hidden state h_j. After softmax normalisation ($\alpha_{t,j}$), a weighted context vector c_t is formed. This allows the decoder to dynamically focus on different source positions.

Topics to Revise

- Bahdanau (additive) vs. Luong (multiplicative) attention
- Alignment matrix visualisation
- Relationship between attention and the Transformer architecture

---

Q 3.3 — Masked Language Modeling (MLM)

Define MLM and explain why it is suitable for bidirectional encoders.

Solution:

Definition: MLM randomly replaces approximately 15% of input tokens with a [MASK] token (and occasionally with a random or unchanged token). The model is trained to predict the original token at each masked position using the surrounding context.

Loss = -sum_{i in masked} log P(x_i | context of all other tokens)

Why it suits bidirectional encoders:

1. The masked token is predicted from BOTH its left and right neighbours simultaneously, which requires bidirectional attention to be meaningful.

2. In autoregressive (left-to-right) models, using the target token as input context would cause trivial label leakage — the model would simply copy the token rather than learn representations.

3. Bidirectional context produces richer, more contextualised representations than unidirectional context, making MLM-pretrained models stronger on understanding tasks.

Topics to Revise

- BERT pre-training: MLM + Next Sentence Prediction (NSP)
- 15% masking strategy: 80% [MASK], 10% random, 10% unchanged
- RoBERTa improvements over BERT (dynamic masking)

Q 3.4 — Soft Prompt-Tuning Parameter Count

Write the formula for trainable prompt-tuning parameters and compute it for m=50, d=768.

Solution:

In prompt tuning, m learnable embedding vectors each of dimension d are prepended to the input. The rest of the LLM is kept frozen.

Trainable parameters = m x d = 50 x 768 = 38,400

Context — why this matters: A full fine-tune of GPT-3 (175B parameters) requires storing and updating 175,000,000,000 gradients. Prompt tuning achieves competitive results by updating only 38,400 parameters — roughly 4.5 million times fewer.

PEFT Method	What is tuned	Typical # parameters
Prompt tuning	Soft prompt embeddings	m x d (tens of thousands)
Prefix tuning	Prefix keys/values per layer	L x 2 x m x d

PEFT Method	What is tuned	Typical # parameters
LoRA	Low-rank weight decompositions	2 x L x r x d (r << d)
Full fine-tune	All model weights	Billions

Topics to Revise

- Prompt tuning (Lester et al. 2021)
- Prefix tuning and LoRA comparison
- PEFT trade-offs: parameter efficiency vs. expressiveness

Q 3.5 — Residual Connections and Layer Normalization

(a) State the Add & Norm formula and explain why it is critical. (b) Compare pre-norm and post-norm.

(a) Add & Norm (Post-norm variant):

output = LayerNorm(x + Sublayer(x))

The residual connection x + Sublayer(x) creates a gradient highway: during backpropagation, gradients can flow directly from the output back to early layers without passing through every sublayer's Jacobian. This prevents the vanishing gradient problem in deep Transformer stacks. Layer normalisation stabilises the distribution of activations across training, accelerating convergence.

(b) Pre-norm vs. Post-norm:

Variant	Formula	Key Advantage
Post-norm (original)	LayerNorm(x + f(x))	Slightly better final task performance; representations at each lay
Pre-norm (modern)	x + f(LayerNorm(x))	More stable training — gradients flow cleanly through the residua

> *Most modern large models (GPT-3, LLaMA, Mistral) use pre-norm because it enables stable training without carefully tuned warmup schedules.*

Topics to Revise

- Vanishing/exploding gradients in deep networks
- Layer norm vs. batch norm
- Pre-norm in LLaMA and GPT-3 architectures

Section 4 — Long Answer Questions (5 marks each)

Q 4.1 — MLM vs. Autoregressive Language Modeling (5 marks)

1. Context Used:

Dimension	MLM (BERT-style)	Autoregressive LM (GPT-style)
Context direction	Bidirectional — sees full left + right context	Unidirectional — only left context at each step
Attention mask	Full attention (no causal mask)	Causal/triangular mask
Pre-training task	Predict masked tokens	Predict the next token
Architecture	Encoder only	Decoder only

2. Loss Computation:

MLM: Cross-entropy loss is computed ONLY at masked positions (~15% of tokens). The model's predictions at non-masked positions do not contribute to the loss.

```
L_MLM = -(1/|M|) * sum_{i in M} log P(x_i | x_{/M})
```

where M is the set of masked positions and x_{/M} denotes all other tokens.

Autoregressive LM: Cross-entropy is computed at EVERY token position. Dense supervision makes training more sample-efficient per sequence.

```
L_AR = -(1/T) * sum_{t=1}^{T} log P(x_t | x_1, ..., x_{t-1})
```

3. Concrete Example:

Input sentence: "The cat sat on the mat"

MLM: Input might be 'The [MASK] sat on the mat' → predict 'cat' at position 2. The model uses 'The', 'sat', 'on', 'the', 'mat' (bidirectional) to predict the missing word.

AR LM: Given 'The cat sat on', predict 'the'. Given 'The cat sat on the', predict 'mat'. Loss accumulated over all 6 positions.

4. Suitable Downstream Tasks:

Task Type	MLM (BERT)	AR LM (GPT)
Classification	Sentiment, spam detection	Chain-of-thought classification
Sequence labelling	NER, POS tagging	Less suited
Span extraction	Extractive QA (SQuAD)	Generative QA
Generation	Not suitable natively	Text generation, dialogue, summarisation, code
Pair tasks	Entailment, similarity	Few-shot via prompting

5. Key Trade-off Summary:

> *MLM builds richer contextual representations because it encodes both directions — better for understanding. AR LM mirrors the generation process and is naturally suited to text generation without any architectural change at inference time.*

Topics to Revise

- BERT paper (Devlin et al. 2019) and GPT-2 paper (Radford et al. 2019)
- T5 and BART: sequence-to-sequence pre-training
- Causal masking implementation in multi-head self-attention
- Evaluation benchmarks: GLUE (BERT) vs. language modelling perplexity (GPT)

Q 4.2 — Reward Model Training and Regularized RLHF Objective (5 marks)

Part (a): Bradley-Terry Loss Computation

The loss for each pair is: l_i = log(sigma(r(x,y+) - r(x,y-))) where sigma(z) = 1/(1+e^{-z})

Pair	r(y+)	r(y-)	Diff = r(y+) - r(y-)	sigma(diff)	l_i = log(sigma)
1	2.0	1.0	1.0	sigma(1.0) = 0.7311	log(0.7311) = -0.3133
2	0.5	1.5	-1.0	sigma(-1.0) = 0.2689	log(0.2689) = -1.3133
3	3.0	0.0	3.0	sigma(3.0) = 0.9526	log(0.9526) = -0.0486

Average loss = -(1/3) * (l_1 + l_2 + l_3) = -(1/3) * (-0.3133 - 1.3133 - 0.0486) = -(1/3) * (-1.6752) = +0.5584

> *Pair 2 contributes by far the largest loss (-1.3133) because the reward model scores the REJECTED response higher than the PREFERRED one — the model got this preference pair completely wrong. This is the most valuable training signal.*

Part (b): Regularized RLHF Objective

The full regularized objective balances reward maximisation with a KL penalty to prevent the policy from deviating too far from the reference (SFT) model:

J(theta) = E_{x~D, y~pi_theta(y|x)} [r(x,y) - beta * log(pi_theta(y|x) / pi_ref(y|x))]

The per-response regularized reward is:

r_s(x,y) = r(x,y) - beta * log(pi_theta(y|x) / pi_ref(y|x))

Numerical computation:

Given: r(x,y) = 1.5, log(pi_theta / pi_ref) = 0.3, beta = 0.1

r_s(x,y) = 1.5 - 0.1 * 0.3 = 1.5 - 0.03 = 1.47

Interpretation:

The regularized reward (1.47) is positive and only slightly below the raw reward (1.5). The KL penalty is small (0.03), indicating the policy has not drifted significantly from the reference model. This response is **beneficial to keep** — it earns positive reward without reward hacking.

> *If log(pi_theta/pi_ref) were very large (e.g., 15), the KL penalty would be 1.5 — completely cancelling the reward — signalling reward hacking. The beta parameter controls this trade-off between task performance and behavioural alignment with the reference model.*

Scenario	r(x,y)	log ratio	KL penalty (beta=0.1)	r_s	Keep?
This case	1.5	0.3	0.03	1.47	Yes — beneficial
Large drift	1.5	15.0	1.50	0.00	Borderline
Reward hacking	2.0	25.0	2.50	-0.50	Suppress

Topics to Revise

- Bradley-Terry preference model and its loss derivation

- RLHF pipeline: SFT -> Reward Model -> PPO
- KL regularisation purpose: preventing reward hacking
- Beta parameter tuning in RLHF
- Constitutional AI and DPO (Direct Preference Optimisation) as alternatives

Master Revision Checklist

Priority topics to study before your exam, grouped by theme:

#	Topic	Questions	Priority
1	Perplexity, cross-entropy, MLE equivalence	1.1, 2.1	High
2	N-gram LMs: Katz backoff, smoothing methods	3.1	High
3	Seq2Seq: architecture, teacher forcing, bottleneck	1.2, 2.2, 3.2	High
4	Attention: Bahdanau, Luong, scaled dot-product	1.5, 3.2	High
5	Transformer: multi-head attention, Add & Norm, pre/post-norm	1.5, 3.5	High
6	BERT vs GPT vs BART pre-training objectives	1.3, 2.3, 3.3, 4.1	High
7	MLM vs Autoregressive LM: tasks, loss, context	3.3, 4.1	High
8	Few-shot / in-context learning, prompt design	1.4	Medium
9	PEFT: prompt tuning, prefix tuning, LoRA	3.4	Medium
10	Instruction tuning, Orca-style data generation	2.4	Medium
11	RLHF: reward model, Bradley-Terry loss	2.5, 4.2	High
12	PPO: on-policy, clipping, sparse rewards	2.5, 4.2	High
13	KL regularisation, beta parameter, reward hacking	4.2	High